Aonde você quer chegar?

Vai com a

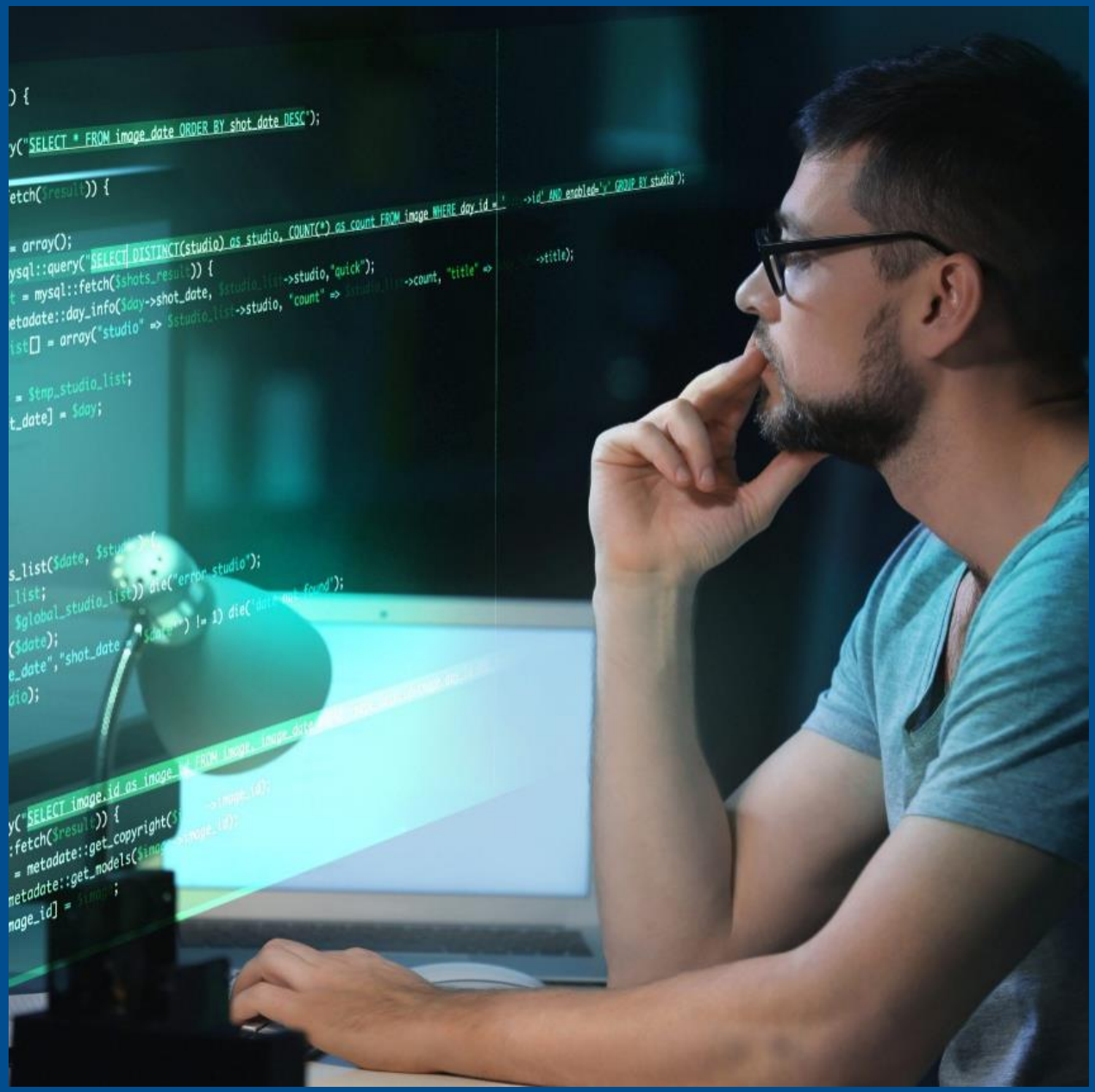
UniCesumar

EAD, SEMIPRESENCIAL E PRESENCIAL

UniCesumar
EAD, SEMIPRESENCIAL E PRESENCIAL

Tecnologias Emergentes em Engenharia de *Software*

Aula 3



Prof. Rômulo Santos

PhD Computação. MSc Engenharia Elétrica. MSc Ciência da Informação.

Sumário

- Camadas de *software* em ambientes industriais.
- Requisitos de qualidade.
- Integração e evidências.

Camadas de *software* em ambientes industriais:

Os ambientes industriais integram diferentes **camadas de *software***, responsáveis por funções específicas, desde o controle de dispositivos físicos até a gestão e a tomada de decisões estratégicas.

Principais camadas e suas funções:

- Dispositivos e controle: sensores, atuadores e controladores (CLPs) responsáveis pela coleta de dados e automação de processos.

- Execução operacional: sistemas SCADA e MES que supervisionam equipamentos, monitoram a produção e gerenciam operações industriais.

- Gestão empresarial: sistemas ERP que integram informações de produção, estoque, finanças e recursos organizacionais.

- Nuvem e integração: plataformas de IoT industrial (IIoT), armazenamento e processamento de dados para conectar sistemas e serviços.

- Apoio à decisão: soluções de *Analytics* e Inteligência Artificial que identificam padrões, realizam previsões e apoiam decisões.

Camadas de *software* em ambientes industriais:

Com o terreno nomeado, a Figura 1 sintetiza as principais categorias e suas relações.

Use-a como referência rápida quando precisar decidir onde medir primeiro. Leia a figura como um radar: o núcleo representa dados/serviços comuns; cada raio, uma categoria típica.

Ao identificar o raio certo, você reduz o ruído, assim, direciona diagnóstico, teste e priorização.

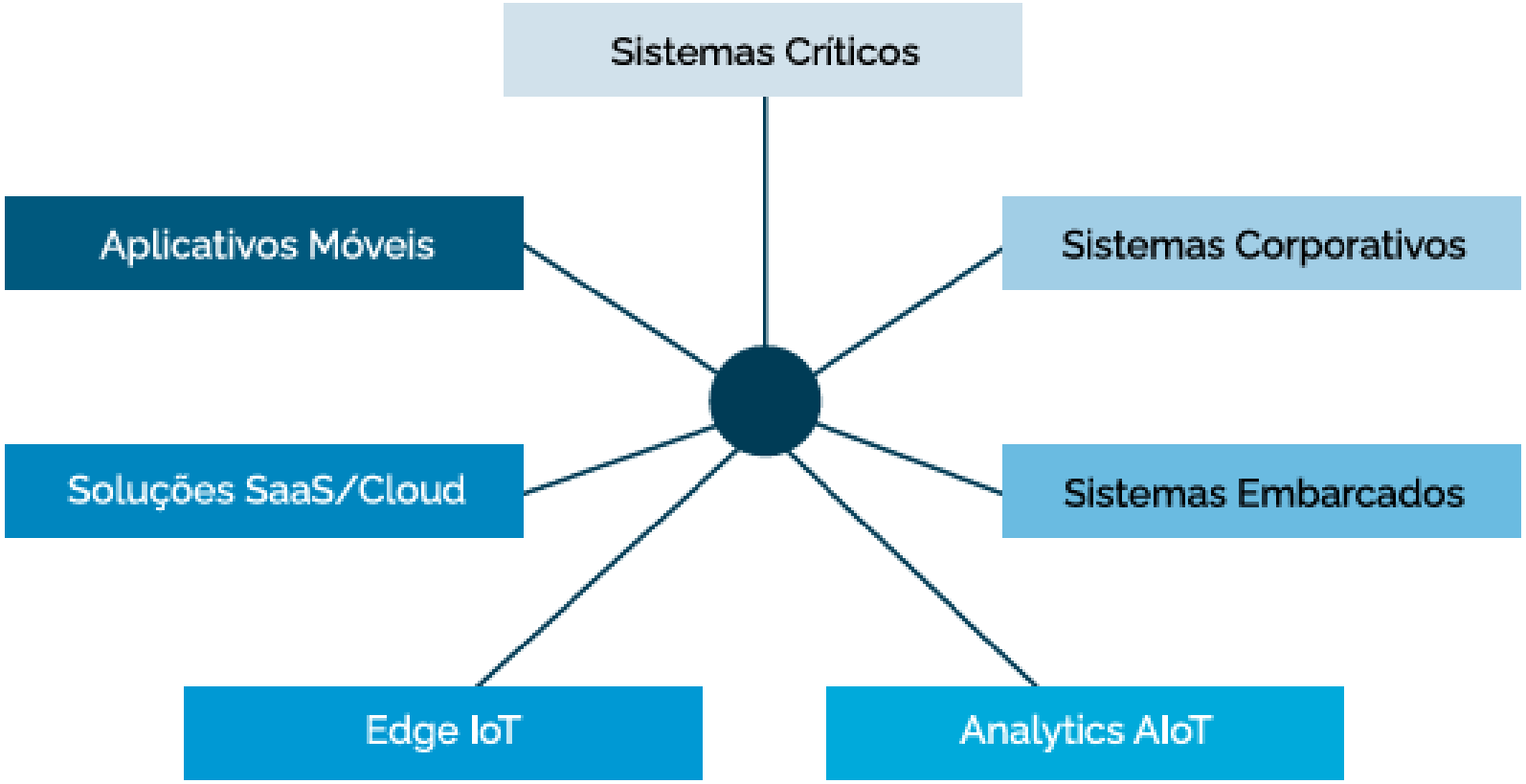


Figura 1 – Classificação de softwares em ambientes industriais (visão geral)
Fonte: gerado por ChatGPT em 21 out. 2025.

Camadas de *software* em ambientes industriais:

Quando alguém diz “o sistema falhou”, a primeira pergunta é: qual sistema? Em empresas industriais e de serviços há, na prática, cinco camadas que convivem e, às vezes, confundem-se.

Há quem viva no dispositivo/na operação, lidando com sensores, atua dores, coletores e apps de campo; quem orquestra a execução/manufatura, registrando etapas, aponta mentos e confirmações; quem sustente a gestão/negócio, com estoque, finanças, manutenção, compras; quem “rode” na nuvem, publicando painéis, APIs e eventos; e quem apoie a decisão, com análises, regras e modelos.

Nomear onde o problema mora muda o jogo, porque cada camada tem prioridades, riscos e métricas diferentes.



Figura 2 – Linha do tempo em sete etapas: Descoberta → Refino → Planejamento → Dev → Testes → Homologação → Release / Fonte: gerado por ChatGPT em 9 nov. 2025.

Camadas de *software* em ambientes industriais:

Quando você identifica, com precisão, onde está o problema no sistema, o ruído cai: a conversa sai de “culpa” e vai para onde medir, o que ajustar e qual evidência comprova.

Essa postura acelera o diagnóstico, qualifica decisões e encurta o ciclo de correção.

Adotar um olhar estratificado por camadas, prioridades e evidências transforma a vaga “o sistema falhou” em um plano de ação claro, com métricas, responsáveis e próximos passos definidos.

DISPOSITIVO/OPERACIONAL

No dispositivo/operacional, a pergunta-chave é: “o comando chega e a resposta volta a tempo?”. Aqui, latência P95, perda de pacotes, estabilidade de rede e energia fazem diferença. Quedas de leitura intermitentes pedem *buffer* local, *retry* exponencial e telemetria no próprio equipamento (Pressman; Maxim, 2016).

EXECUÇÃO/MANUFATURA

Na execução/manufatura, o que vale é disponibilidade por turno e confirmação confiável: ordens não podem “sumir”. Medimos *uptime* por célula, taxa de confirmação, filas crescentes e tempos “emissão → confirmação”. Um plano B simples, **fila local + reenvio**, costuma reduzir picos de indisponibilidade sem parar a linha (Fernandes; Teixeira, 2011).

Camadas de *software* em ambientes industriais:

Quando você identifica, com precisão, onde está o problema no sistema, o ruído cai: a conversa sai de “culpa” e vai para onde medir, o que ajustar e qual evidência comprova.

Essa postura acelera o diagnóstico, qualifica decisões e encurta o ciclo de correção.

Adotar um olhar estratificado por camadas, prioridades e evidências transforma a vaga “o sistema falhou” em um plano de ação claro, com métricas, responsáveis e próximos passos definidos.

GESTÃO/NEGÓCIO

Subindo para a gestão/negócio, o risco é inconsistência: um estoque que diverge, uma nota que não concilia. Métricas como reconciliações por período, volume de ajustes manuais e tempo de acerto apontam se o processo é confiável. Trilha de auditoria (“quem mudou o que, quando e por quê”), **idempotência** nas integrações e validações de domínio antes de gravar dado diminuem retrabalho (ISO, 2017; Pressman; Maxim, 2016).

NUVEM/SERVIÇOS

Na nuvem/serviços, a conversa é observabilidade e metas de serviço: SLIs/SLOs de latência e erro, taxa de sucesso em integrações, saúde de filas e *backoffs*. Painéis claros, alertas com bom sinal/ruído e *feature flags* para ativar/desativar funcionalidades dão segurança para evoluir sem big bang (Pressman; Maxim, 2016; Sommerville, 2018).

APOIO À DECISÃO

Por fim, no apoio à decisão, importa qualidade e linhagem de dados. De onde veio, como foi transformado, com que frescor e se houve drift. Medimos completude, atualidade, acurácia amostral e disparamos avisos quando um modelo sai da faixa esperada (Sommerville, 2018). Sem isso, um gráfico bonito pode orientar mal a operação. Perceba o padrão: em cada camada, existe uma pergunta mestra, um risco dominante e um conjunto mínimo de evidências. Se o painel não atualiza, olhamos ingestão e eventos (nuvem); se a ordem não confirma, investigamos fila e retry (execução); se o saldo não fecha, rastreamos integrações e conciliações (gestão); se o sensor "pula", medimos P95 e perda de pacotes (dispositivo); se a previsão está errando mais, verificamos drift e amostra (análises).

Estudo de caso 1:
Falha em uma linha de produção industrial
Diagnóstico de problemas utilizando as camadas de software



Objetivo
Compreender a relação entre as camadas de dispositivos, execução operacional, gestão, nuvem e apoio à decisão.

Cenário
Uma fábrica apresenta interrupções na produção.
O CLP registra falhas de comunicação com sensores, o MES não confirma ordens e o ERP apresenta divergências de estoque.

Desafio para os alunos
- Identificar a possível origem das falhas.
- Determinar quais sistemas foram afetados.
- Selecionar métricas adequadas para investigação.

Camadas envolvidas
1. **Dispositivos:** sensores e atuadores
2. **Execução operacional:** CLP e MES
3. **Gestão:** ERP
4. **Nuvem/monitoramento**
5. **Apoio à decisão:** análise de indicadores

Métricas sugeridas
- Perda de pacotes
- Latência
- Disponibilidade
- Taxa de confirmação de ordens
- Tempo de parada da linha

Resultado esperado
Demonstrar como o diagnóstico orientado por camadas evita atribuir indiscriminadamente a falha ao sistema inteiro.

Perguntas norteadoras
1. Onde a falha se iniciou?
2. Quais evidências confirmam o problema?
3. Como priorizar a correção?

Diagrama das camadas (visão simplificada)



Análise por camadas permite identificar a **verdadeira causa, reduzir o tempo de parada** e **aumentar** a **confiabilidade** da produção.

Requisitos de qualidade:

A avaliação da qualidade de *software* deve utilizar **indicadores mensuráveis**, permitindo identificar problemas, estabelecer prioridades e fundamentar decisões técnicas.

Cinco dimensões fundamentais de avaliação:

- Disponibilidade: medir o tempo de indisponibilidade do sistema, distinguindo paradas planejadas de falhas inesperadas.

- Desempenho (P95): monitorar o tempo de resposta abaixo do qual estão 95% das requisições, estabelecendo metas de desempenho (SLOs).

- Integridade e rastreabilidade: relacionar requisitos, alterações no código, testes e versões para garantir histórico e evidências de validação.

- Segurança e privacidade: aplicar menor privilégio, criptografia, gestão de segredos e verificações automatizadas de vulnerabilidades.

- Observabilidade: utilizar logs, métricas e rastreamento distribuído (*tracing*) para identificar falhas e monitorar o comportamento do sistema.

Prioridades conforme o contexto: saúde (confidencialidade), indústria (resposta rápida), agronegócio (operação *off-line*) e logística (latência e integridade dos eventos).

Requisitos de qualidade:

Para tornar essas lentes mais práticas, vale observar como o contexto muda conforme o tipo de *software*. A natureza do produto (embarcado, corporativo, móvel, crítico, nuvem, apoio à decisão) determina quais lentes pesam mais e quais evidências precisam ser coletadas, bem como acompanhadas.

Agora que você entende “onde” vivem os sistemas, vale olhar “que tipo” de *software* está em jogo. O tipo ajusta o peso das lentes e orienta o que medir e provar (Sommerville, 2018), como veremos a seguir.

EMBARCADO
Opera em dispositivos; exige tempo de resposta e confiabilidade (IEC, 2006; RTCA, 2011).
CORPORATIVO (GESTÃO)
Processos e dados; pede integridade e auditoria.
CRÍTICO
Erro = risco elevado; requer evidências, validação independente e redundância (IEC, 2010; ISO, 2017; RTCA, 2011).
MÓVEL INDUSTRIAL
Campo e off-line; demanda sincronização robusta e UX em condições adversas.
SERVIÇO EM NUVEM
Multiusuário; precisa de observabilidade, SLO/SLI e segurança por padrão (Pressman; Maxim, 2016).
APOIO À DECISÃO
Dados/modelos; foca qualidade de dados, explicabilidade e ciclo de vida do modelo.

Estudo de caso 2:

Sistema de monitoramento agrícola com conectividade limitada

Aplicação dos requisitos de qualidade em condições adversas



Objetivo

Aplicar os cinco requisitos de qualidade a uma solução que opera em condições adversas.

Cenário

Uma empresa agrícola utiliza sensores IoT e aplicativos móveis para registrar temperatura, umidade e operações de campo. A perda frequente de conectividade provoca atrasos e inconsistências na sincronização.

Desafio para os alunos

- Propor mecanismos de operação off-line.
- Definir estratégias de reconciliação de dados.
- Garantir rastreabilidade e segurança.
- Estabelecer indicadores de disponibilidade, latência P95 e integridade.

Requisitos de qualidade prioritários

1. **Disponibilidade:** operação mesmo com rede instável.
2. **Desempenho:** sincronização com P95 controlado.
3. **Integridade e rastreabilidade:** registro confiável das operações.
4. **Segurança e privacidade:** proteção dos dados em campo e na nuvem.
5. **Observabilidade:** logs, métricas e evidências de sincronização.

Mecanismos recomendados

- Modo off-line com armazenamento local
- Fila de sincronização e retries
- Reconciliação e deduplicação de registros
- Identificadores únicos para rastreabilidade
- Criptografia e gestão de credenciais
- Painéis de monitoramento e alertas

Métricas sugeridas

- Disponibilidade do app e dos sensores
- Latência P95 da sincronização
- Taxa de conflitos de dados
- Percentual de registros reconciliidos
- Falhas de autenticação / segurança

Resultado esperado

Compreender que os requisitos de qualidade precisam ser priorizados conforme o contexto operacional do software.

Fluxo simplificado

Sensores
Temperatura, umidade e operações de campo

→

App móvel
Coleta e operação off-line

→

Fila local
Armazenamento e retries de sincronização

→

Nuvem
Reconciliação e deduplicação

→

Painel analítico
Visualização, métricas e alertas

Perguntas norteadoras

1. Onde está o maior risco operacional?
2. Como garantir sincronização confiável?
3. Quais evidências comprovam a qualidade?

Qualidade em campo exige resiliência, sincronização confiável e evidências mensuráveis.

Estudo de caso 3: Pedidos duplicados na integração entre sistemas

Contratos, idempotência e tratamento de falhas em um cenário de e-commerce

Tecnologias Emergentes em Engenharia de Software

ARQUITETURAS • INTEGRAÇÃO • RESILIÊNCIA

Objetivo

Demonstrar a importância dos contratos de integração, da idempotência e do tratamento de falhas.

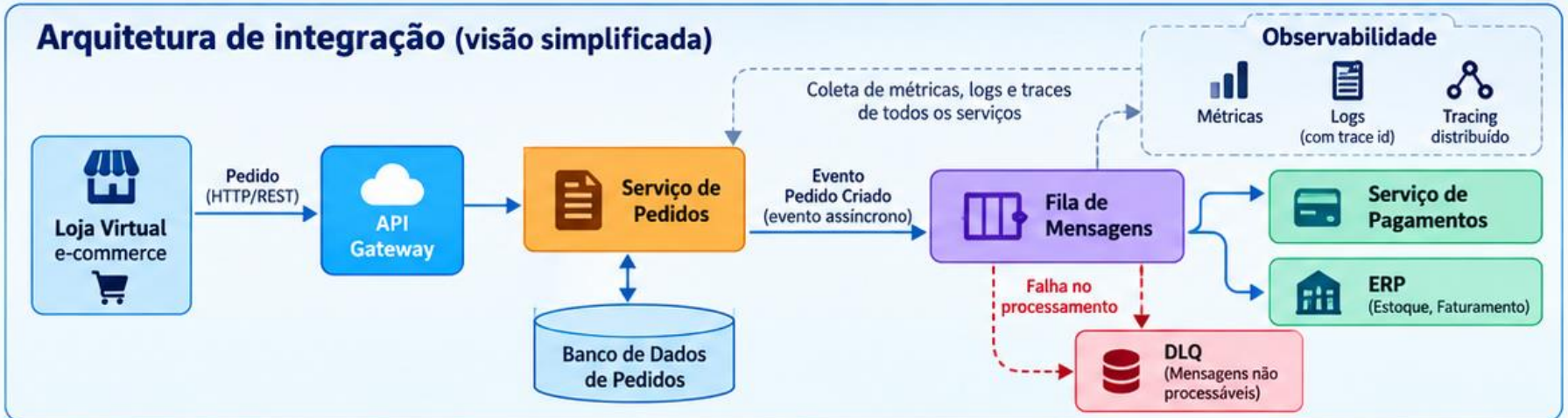
Cenário

Uma empresa de comércio eletrônico integra seu sistema de pedidos a um ERP e a um serviço de pagamento. Uma falha de comunicação provoca o reenvio de mensagens, ocasionando cobranças e registros duplicados.

Desafio para os alunos

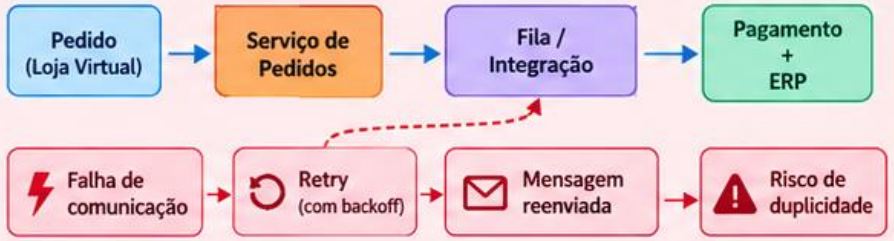
- Definir uma chave de idempotência por pedido
- Estabelecer políticas de retry e backoff
- Usar DLQ para mensagens não processáveis
- Implementar testes de contrato e regressão
- Monitorar duplicidade, erro e latência

Arquitetura de integração (visão simplificada)

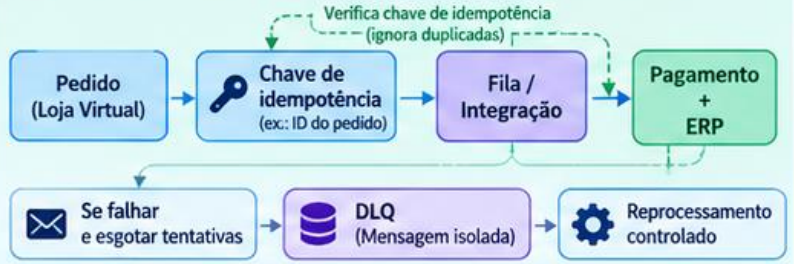


Fluxo da falha e tratamento

Cenário problemático (pode gerar pedidos duplicados)



Cenário com boas práticas (evita cobrança duplicada)



Elementos essenciais da solução

- Contratos versionados (OpenAPI / eventos)
- Idempotência e deduplicação
- Retries com backoff e timeouts coerentes
- DLQ e reprocessamento controlado
- Logs com trace id, métricas e tracing

Métricas sugeridas

- Taxa de pedidos duplicados
- Latência P95 da integração
- Taxa de erro por integração
- Mensagens enviadas para DLQ
- Tempo de reconciliação

Resultado esperado

Demonstrar que a integração robusta precisa suportar falhas, mensagens duplicadas e mudanças de contrato sem comprometer a consistência dos dados.

Sistemas resilientes mantêm a integridade dos dados mesmo diante de falhas.

Integração confiável exige contrato testado, idempotência, observabilidade e evidências automatizadas.

Integração e evidências:

A integração de sistemas deve garantir uma **comunicação confiável, previsível e segura**, mesmo diante de falhas, atrasos ou mudanças nas aplicações.

Principais práticas de integração:

- Contratos versionados: definir formatos de dados, campos obrigatórios, códigos de erro e regras de compatibilidade, utilizando OpenAPI, JSON ou Avro.

- Testes de contrato: validar automaticamente a comunicação entre sistemas nos pipelines CI/CD, identificando incompatibilidades antes da implantação.

- Idempotência: garantir que a repetição de uma mesma operação não produza efeitos indesejados, como pagamentos duplicados.

- Tratamento de falhas: implementar *timeouts*, *retries* com *backoff* e filas de mensagens não processadas (DLQ).

- Ordenação e deduplicação: controlar mensagens duplicadas, atrasadas ou fora de sequência, garantindo a consistência das operações.

Estudo de caso 4:

Falha após implantação de uma nova versão

Observabilidade, SLI/SLOs e evidências automatizadas



Contexto

Uma nova versão de uma API foi implantada em produção. Após o deploy, o tempo de resposta P95 aumentou de 1,2 s para 4 s e a taxa de erro chegou a 1,5%, ultrapassando os SLOs estabelecidos (P95 ≤ 2 s e erro ≤ 0,5%).

Desafio

- Investigar logs, métricas e traces para identificar a causa.
- Localizar o serviço responsável pelo aumento de latência e erros.
- Decidir sobre rollback ou correção.
- Identificar quais evidências devem ser preservadas no pipeline CI/CD.

Objetivo

Aplicar observabilidade, SLI/SLO, testes pós-implantação e Evidence as Code para apoiar decisões de estabilização, correção e evolução segura dos sistemas.

Resultado esperado

Compreender como a observabilidade e a rastreabilidade apoiam decisões de estabilização, correção e evolução segura dos sistemas, evitando impacto ao usuário e garantindo evidências técnicas da entrega.

Arquitetura e cenário

Métricas (antes e depois do deploy)



Rastreamento distribuido (trace)



Trecho de logs (exemplo)

```
2026-10-05T14:21:33.453Z  ERROR  [servico-a]  Timeout ao consultar serviço B
2026-10-05T14:21:33.453Z  WARN   [servico-a]  Retry attempt 1/3
2026-10-05T14:21:35.789Z  ERROR  [servico-a]  Connection pool exhausted
2026-10-05T14:21:35.790Z  INFO   [servico-a]  Fallback ativado
2026-10-05T14:21:37.120Z  ERROR  [servico-a]  Request failed after 3 retries
```

Pipeline CI/CD e evidências



Integração e evidências:

A qualidade das integrações deve ser continuamente monitorada e comprovada por meio de **métricas, testes e evidências geradas automaticamente**.

Principais mecanismos de controle:

- Observabilidade: utilizar *logs* estruturados, métricas e rastreamento distribuído (*tracing*) para identificar falhas e acompanhar a comunicação entre serviços.

- Indicadores SLIs/SLOs: monitorar latência P50/P95, taxa de erros e *throughput*, estabelecendo metas como P95 ≤ 2 segundos e erros ≤ 0,5%.

- Validação pós-implantação: executar *smoke tests* e *health checks* para verificar se as integrações permanecem operacionais.

- *Evidence as Code*: gerar automaticamente relatórios de testes, verificações de segurança e registros de alterações durante o pipeline CI/CD.

- Rastreabilidade: relacionar requisitos, *issues*, *pull requests*, pipelines, *releases* e indicadores operacionais.

- Melhoria contínua: realizar análises de incidentes (*post-mortem*) para identificar causas, corrigir falhas e prevenir recorrências.

Bons estudos!

Siga em frente com a



www.unicesumar.edu.br